utad

Relatório – ProblemSet 2

O Caminho Mais Rápido por Hyrule

Introdução ao Pensamento Computacional e Ciência de Dados

Autores:

Francisco Veiga (al82066) · Marco Maceda (al66694)

Docentes:

Arsénio Monteiro dos Reis · Ricardo Rodrigues Nunes

1. Introdução

O problem set que propusemos é uma adaptação do Problem Set 2 do MIT (*Fastest Way to Get Around MIT*). No original, o objetivo é encontrar o caminho mais curto entre dois edifícios do campus, limitando a distância percorrida no exterior. Na nossa versão, o mapa é o reino de Hyrule, de *The Legend of Zelda: Breath of the Wild*: os 120 santuários do jogo são os nós de um grafo dirigido, e o limite de distância no exterior passa a ser um limite de perigo, porque algumas regiões do mapa são mais perigosas do que outras.

Este relatório descreve, passo a passo, o que fizemos: onde os dados foram obtidos, como construímos o grafo, como implementámos cada problema (estruturas de dados, carregamento do mapa e pesquisa em profundidade) e o programa final que sorteia dois santuários e compara a rota mais rápida com a rota mais rápida e segura. Termina com um resumo das diferenças face ao problema do MIT e com as limitações do trabalho.

Para facilitar a leitura, chamamos Problema 1 às estruturas de dados (graph.py), Problema 2 ao carregamento do mapa (load_map) e Problema 3 à pesquisa do caminho mais curto (get_best_path e directed_dfs), tal como no enunciado do MIT.

2. Origem dos dados

Ficheiro	O que contém	Script
shrines.csv	id, nome, região e posição (x, y) de cada um dos 120 santuários	scrape_shrines.py e mark_shrines.py
botw_map.jpg	imagem do mapa de Hyrule, usada para marcar as posições e para desenhar o grafo	–
edges.csv	arestas dirigidas: from, to, distance, danger_level	edges_creator.py

Lista de santuários e regiões. O scrape_shrines.py extrai a lista dos 120 santuários da página de santuários do Thonky.com (com requests e BeautifulSoup). Cada região tem um título do tipo *X Region Shrines* seguido de uma lista, e o nome do santuário é separado do resto do texto com uma expressão regular. O script avisa se o número de santuários for diferente de 120.

Posições no mapa. A página não tem coordenadas, por isso as posições foram marcadas por nós, uma a uma, clicando sobre a imagem do mapa com o mark_shrines.py (plt.ginput). O script grava o ficheiro a cada clique e pode ser interrompido e retomado: só pede os santuários que ainda não têm posição. As coordenadas são em píxeis da imagem (origem no canto superior esquerdo, y a crescer para baixo).

O ficheiro final tem o formato `id,name,region,x,y`.

3. Como construímos o grafo (edges_creator.py)

O edges_creator.py lê o shrines.csv e escreve o edges.csv. Os parâmetros principais são:

```
K = 2                    # vizinhos mais próximos de cada santuário
MAX_DIST = 200.0         # comprimento máximo de uma aresta, em píxeis
NO_REVERSE_DUPLICATES = True
SEED = 0                 # torna a escolha aleatória reproduzível
```

O processo tem quatro passos:

1. **Vizinhos mais próximos.** Cada santuário recebe uma aresta de saída para cada um dos seus K vizinhos mais próximos, desde que estejam a menos de MAX_DIST píxeis.
2. **Sem arestas inversas duplicadas.** Quando existem A→B e B→A, mantém-se só uma delas, escolhida ao acaso (com SEED fixa). Assim o grafo é realmente dirigido: ir de A para B pode ser possível sem que o contrário o seja.
3. **Grafo fortemente conexo.** Para que exista sempre um caminho entre quaisquer dois santuários, o script calcula as componentes fortemente conexas (com pesquisas em profundidade) e, enquanto houver mais do que uma, acrescenta arestas de ligação entre componentes. As ligações são acrescentadas num só sentido de cada vez, escolhendo a mais curta que feche um ciclo; só em último caso se acrescentam os dois sentidos.
4. **Dois pesos por aresta.** A distância é a distância euclidiana, em píxeis, entre os dois santuários. O nível de perigo é o número de extremos da aresta (0, 1 ou 2) que estão numa região perigosa.

```
DANGEROUS_REGIONS = {
    'Ridgeland', 'Tabantha', 'Hebra', 'Wasteland', 'Eldin', 'Central', 'Faron',
}

def danger_level(a, b, nodes):
    return (int(nodes[a].region in DANGEROUS_REGIONS)
            + int(nodes[b].region in DANGEROUS_REGIONS))
```

O resultado é um grafo com **120 nós e 233 arestas dirigidas**. O script show_map.py desenha-o sobre o mapa, com a cor de cada aresta a indicar o nível de perigo (verde 0, laranja 1, vermelho 2).

4. Problema 1: estruturas de dados (graph.py)

Usámos as classes Node, Edge e Digraph fornecidas e completámos a WeightedEdge e os métodos add_node e add_edge do Digraph.

```
class WeightedEdge(Edge):
    def __init__(self, src, dest, distance, danger_level):
        super().__init__(src, dest)
        self.distance = distance
        self.danger_level = danger_level

    def get_distance(self):
        return self.distance

    def get_danger_level(self):
        return self.danger_level

    def __str__(self):
        return f'{self.src}->{self.dest} ({self.distance}, {self.danger_level})'
```

```
def add_node(self, node):
    if node in self.nodes:
        raise ValueError('Node already exists')
    self.nodes.add(node)
    self.edges[node] = []

def add_edge(self, edge):
    src = edge.get_source()
    dest = edge.get_destination()
    if src not in self.nodes or dest not in self.nodes:
        raise ValueError('Node does not exist')
    self.edges[src].append(edge)
```

O add_node cria logo a lista de arestas do nó, para que o add_edge e a pesquisa possam usar `self.edges[node]` sem verificações extra. Os dois métodos lançam ValueError nos casos pedidos no enunciado (nó repetido e aresta com um nó que não está no grafo).

Os testes unitários do fim do graph.py foram adaptados aos nomes que escolhemos (get_distance e get_danger_level, em vez de get_total_distance e get_outdoor_distance) e **os 6 testes passam**.

5. Problema 2: construir o mapa

2a. Modelação

- **Nós:** cada santuário é um Node, identificado pelo seu id (uma string, por exemplo '33').
- **Arestas:** cada linha do edges.csv é uma WeightedEdge dirigida, que representa uma ligação que o Link pode percorrer de um santuário para outro.
- **Pesos:** a distância (em píxeis do mapa) e o nível de perigo (0, 1 ou 2) ficam guardados na própria aresta.
- **Porque é dirigido:** as arestas só podem ser percorridas no sentido indicado, e como removemos as arestas inversas duplicadas, ir de A para B pode ser diferente de ir de B para A.

2b. load_map

O load_map lê os dois ficheiros CSV: primeiro cria um nó por santuário e depois uma aresta por linha do edges.csv. A distância é convertida para número decimal e o perigo para inteiro. Além do grafo, guarda num dicionário o nome, a região e a posição de cada santuário, que serve para mostrar os resultados e desenhar o mapa.

```
def load_map(nodes_file=NODES_FILE, edges_file=EDGES_FILE):
    g = Digraph()
    info = {}
    with open(nodes_file, newline='', encoding='utf-8') as f:
        for row in csv.DictReader(f):
            g.add_node(Node(row['id']))
            info[row['id']] = {'name': row['name'], 'region': row['region'],
                               'x': float(row['x']) if row['x'] else None,
                               'y': float(row['y']) if row['y'] else None}
    with open(edges_file, newline='', encoding='utf-8') as f:
        for row in csv.DictReader(f):
            g.add_edge(WeightedEdge(Node(row['from']), Node(row['to']),
                                    float(row['distance']),
                                    int(row['danger_level'])))
    return g, info
```

2c. Verificação

Com o mapa completo, o grafo carregado tem **120 nós e 233 arestas**, o que coincide com o que o edges_creator.py escreveu. O programa imprime estes números sempre que corre.

6. Problema 3: caminho mais curto com pesquisa em profundidade

3a. Função objetivo

Elemento	Descrição
Função objetivo	Minimizar a distância total do caminho
Restrição principal	Perigo total do caminho ≤ limite de perigo
Restrição opcional	Distância total ≤ distância máxima

3b. get_best_path

A função segue a estrutura do enunciado do MIT: uma pesquisa em profundidade recursiva que constrói os caminhos possíveis a partir do nó de partida. O caminho atual é uma lista com os nomes dos nós, a distância acumulada e o perigo acumulado. Há três cortes na pesquisa:

- **Ciclos:** não se visita um nó que já está no caminho atual.
- **Perigo:** abandona-se o caminho assim que o perigo acumulado excede o limite.
- **Poda por distância:** abandona-se o caminho assim que a distância acumulada já não melhora o melhor caminho encontrado, porque os seus descendentes também não o melhorariam.

```
def get_best_path(digraph, start, end, path, max_danger, best_dist, best_path):
    if start == end:
        return (path[0][:], path[1])

    for edge in digraph.get_edges_for_node(Node(start)):
        nxt = edge.get_destination().get_name()
        if nxt in path[0]:                      # evita ciclos
            continue
        new_dist = path[1] + edge.get_distance()
        new_danger = path[2] + edge.get_danger_level()
        if new_danger > max_danger:             # restrição de perigo
            continue
        if best_path is None:                   # ainda sem solução:
            if new_dist > best_dist:            #   só o limite de distância
                continue
        elif new_dist >= best_dist:             # poda: já não melhora
            continue
        result = get_best_path(digraph, nxt, end,
                               [path[0] + [nxt], new_dist, new_danger],
                               max_danger, best_dist, best_path)
        if result is not None:
            best_path, best_dist = result
    return (best_path, best_dist) if best_path is not None else None
```

3c. directed_dfs

A função directed_dfs só inicializa as variáveis, chama a função recursiva e devolve o caminho. O limite de distância serve de valor inicial de best_dist. Se nenhum caminho cumprir as restrições, lança ValueError.

```
def directed_dfs(digraph, start, end, max_total_dist, max_danger):
    if not digraph.has_node(Node(start)) or not digraph.has_node(Node(end)):
        raise ValueError('Nó inexistente')
    result = get_best_path(digraph, start, end, [[start], 0, 0],
                           max_danger, max_total_dist, None)
    if result is None:
        raise ValueError('Nenhum caminho cumpre as restrições')
    return result[0]
```

7. Comparar duas rotas entre santuários aleatórios (rota_hyrule.py)

O rota_hyrule.py reúne tudo num só programa, usando as classes do graph.py. Faz o seguinte:

1. **Sorteia** um santuário de partida e um de destino diferentes (com random, e opcionalmente uma semente para o resultado ser reproduzível, ou então com o par indicado por nós).
2. **Rota mais rápida:** chama directed_dfs sem limite de perigo.
3. **Rota mais rápida e segura:** procura o menor perigo possível e, com esse perigo, a menor distância.
4. **Mostra e desenha** as duas rotas sobre o mapa.

7.1 Como obtemos a rota mais rápida e segura

O perigo total de um caminho é um número inteiro, e se existe um caminho com perigo máximo P, também existe para qualquer limite maior. Por isso, uma pesquisa binária sobre o limite de perigo encontra o menor limite para o qual o directed_dfs ainda devolve um caminho. Esse limite é o perigo mínimo possível entre os dois santuários. Com ele, o directed_dfs devolve o caminho mais curto de entre os mais seguros. O perigo da rota mais rápida serve de limite superior, porque é sempre viável.

```
def fastest_safest_path(g, start, end, danger_of_fastest):
    lo, hi = 0, int(danger_of_fastest)        # hi é sempre viável
    while lo < hi:
        mid = (lo + hi) // 2
        try:
            directed_dfs(g, start, end, NO_LIMIT, mid)
            hi = mid                          # há caminho: tenta menos perigo
        except ValueError:
            lo = mid + 1                      # não há: precisa de mais perigo
    return directed_dfs(g, start, end, NO_LIMIT, lo)
```

Definimos portanto a rota mais rápida e segura como a de menor perigo e, dentro dessas, a de menor distância. Esta é uma escolha nossa: outra definição possível seria aceitar mais perigo em troca de uma rota bastante mais curta.

7.2 Visualização

O programa desenha as duas rotas sobre a imagem do mapa: a rota mais rápida a vermelho (linha mais grossa) e a rota mais rápida e segura a verde por cima. A partida é marcada com uma estrela e o destino com uma cruz. Quando as duas rotas coincidem, a verde passa a tracejada e o título do mapa avisa.

7.3 Utilização

```
python rota_hyrule.py --different          # só pares com rotas diferentes
python rota_hyrule.py --seed 7             # par aleatório reproduzível
python rota_hyrule.py --start 33 --end 16  # par escolhido por nós
python rota_hyrule.py --save rota.png      # grava a imagem
```

O programa precisa do graph.py na mesma pasta e dos ficheiros data/shrines.csv, data/edges.csv e data/botw_map.jpg, e de ter o matplotlib instalado.

7.4 Resultados

Par de santuários	Rota	Passos	Distância (px)	Perigo
Daag Chokah (33) → Tahno O'ah (16)	Mais rápida	24	2414,2	18
	Mais rápida e segura	29	2993,4	12
Monya Toma (31) → Rok Uwog (77)	As duas rotas coincidem	9	728,3	17

No primeiro par, ser mais seguro custa mais 579,2 px (e mais 5 passos) para reduzir o perigo de 18 para 12. No segundo, a rota mais rápida já é a menos perigosa.

Numa experiência com 40 pares aleatórios, as duas rotas foram diferentes em 7 (cerca de 1 em cada 6). Nos restantes, o caminho mais curto já era o de menor perigo, o que é natural num grafo com poucas alternativas por santuário (233 arestas para 120 nós). Por isso o programa tem a opção --different, que sorteia pares até as rotas serem diferentes.

8. Resumo das alterações face ao Problem Set do MIT

Aspeto	Problem Set do MIT	Nossa versão	Porquê
Mapa	Edifícios do campus (mit_map.txt: 37 nós, 129 arestas)	120 santuários de Breath of the Wild (120 nós, 233 arestas)	Tema novo para o problema proposto
Segundo peso	Distância no exterior	Nível de perigo (0, 1 ou 2)	Mantém a estrutura: dois pesos por aresta, uma função objetivo e uma restrição
Ficheiros de entrada	Um ficheiro de texto com 4 números por linha	Dois CSV (santuários e arestas)	Precisamos do nome, da região e da posição para mostrar e desenhar
Construção dos dados	Ficheiro fornecido	Scripts de extração, marcação manual das posições e criação das arestas	Os dados tiveram de ser construídos de raiz
Sentido das arestas	Dirigidas, quase sempre nos dois sentidos	Dirigidas, sem arestas inversas duplicadas	Faz com que o sentido importe
WeightedEdge	total_distance e outdoor_distance	distance e danger_level (get_distance e get_danger_level)	Nomes de acordo com o novo significado
load_map	Lê um ficheiro e devolve o grafo	Lê dois CSV e devolve o grafo e a informação dos santuários	Mostrar nomes e desenhar o mapa
Restrição	Distância máxima no exterior	Perigo máximo	O perigo substitui o frio
directed_dfs	Devolve o caminho mais curto ou lança ValueError	Igual, com o limite de perigo	Mesma ideia, outro peso
Programa final	Testes unitários	Sorteio de pares, rota mais rápida, rota mais rápida e segura (pesquisa binária) e desenho no mapa	Comparar o efeito do limite de perigo

9. Limitações

- **Distâncias em píxeis.** A distância é medida na imagem do mapa e não em unidades do jogo. Como o mapa é uma imagem plana, serve para comparar rotas entre si, mas não corresponde a um tempo de viagem real.
- **Perigo simplificado.** O nível de perigo só depende da região dos dois extremos da aresta (e da lista de regiões que escolhemos como perigosas). Não distingue, por exemplo, uma zona calma e uma zona hostil dentro da mesma região.
- **Grafo artificial.** As arestas vêm de uma regra (os 2 vizinhos mais próximos, com ligações extra para garantir conectividade) e não dos caminhos reais do jogo. A escolha do sentido de cada aresta é aleatória, controlada por uma semente.

- **Tempo de execução.** A pesquisa em profundidade com poda pode explorar muitos caminhos no pior caso. Nos testes que fizemos terminou sempre, mas não medimos tempos nem comparámos com outros algoritmos.
- **Definição de rota segura.** Escolhemos minimizar primeiro o perigo e depois a distância; outras escolhas dariam rotas diferentes.

10. Conclusão

Adaptámos o problema do MIT a um mapa que construímos nós, com dois pesos por aresta e uma restrição de perigo no lugar da distância no exterior. Completámos as estruturas de dados (WeightedEdge e Digraph), carregámos o mapa a partir de dois ficheiros CSV e implementámos a pesquisa em profundidade com as duas otimizações do enunciado (evitar ciclos e podar caminhos que já não melhoram o melhor encontrado).

O programa final mostra o efeito do limite de perigo: em alguns pares de santuários, a rota mais segura obriga a um desvio longo, como se viu no exemplo em que o perigo baixou de 18 para 12 à custa de mais 579 píxeis, e noutros a rota mais rápida já é a mais segura. A pesquisa binária sobre o limite de perigo permitiu obter a rota mais rápida e segura usando apenas a função directed_dfs pedida no problema.